

改良版ソーラーカーパッケージ オールインワン解説書兼体験ワーク

solar_ws0129-main

2026-07-16

本書の目的

本書は、現在の改良版ソーラーカーパッケージについて、

- PowerShell 入口から何が起動するか
- offline simulation / live / live_wifi / measure / forecast / identify がどのように繋がるか
- 実測ログからの MLE 同定がどの数式で行われ、どの `profile` / `map` / `planner` へ反映されるか
- WiFi 文字列テレメトリをソーラーカー側・伴走車側でどう送受信すればよいか
- 現在の `maps` / `coefficients` の意味と所在

を一冊で追えるようにまとめたものである。

PASSO 系列と磁力関連は本書の主対象から外し、ソーラーカー本体の `planner` / `sim` / `live` 運用に集中する。

重要な結論: 現在の `package` の本質は PowerShell / `shell` の入口、`launch`、`mpc_solarcar/*.py`、`offline sim`、車両同定、各 `profile.yaml` にある。本書では、この入口から物理モデル、同定、運用までを一貫して説明する。

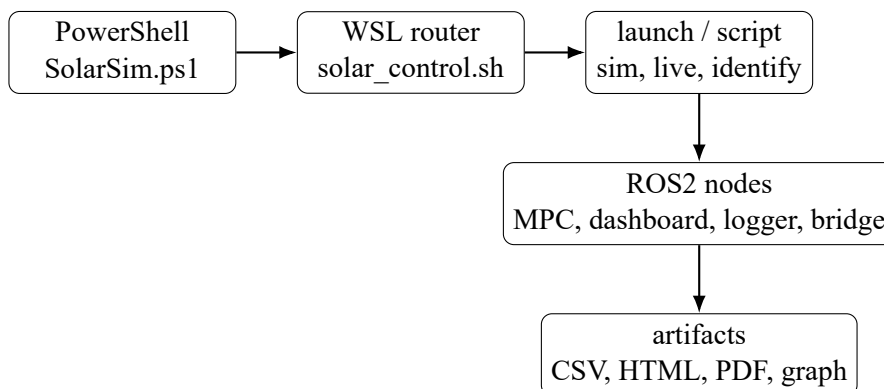
Contents

1	まず最初に全体像	3
1.1	一番上の流れ	3
1.2	PowerShell で叩くもの	3
1.3	現在の source 側で可視になっている入口	3
1.4	確認問題 1	4
2	profile.yaml の読み方	5
2.1	top-level 構造	5
2.2	この profile がどこに効くか	5
2.3	既定 profile	5
3	mode 別に何が起きるか	6
3.1	sim	6
3.2	measure	6

3.3	live	6
3.4	live_wifi	7
3.5	launch file ごとの実起動 node	7
4	物理モデル	8
4.1	使っている基本式	8
4.2	実装でこれを使っている場所	9
4.3	確認問題 2	9
5	実測ログからの MLE / fitting	9
5.1	pipeline	9
5.2	軽量 pipeline	9
5.3	本 package の本格 replay fit	10
5.4	fit した値はどこへ入るか	10
5.5	どのファイルを入れれば generic fitting が回るか	11
6	一画面 dashboard の読み方	11
7	live_wifi の送受信仕様	12
7.1	推奨方式	12
7.2	ソーラーカー側から送る推奨 JSON	12
7.3	伴走車側から送る推奨 JSON	13
7.4	planner 側から返す JSON	13
7.5	key=value fallback	13
7.6	マイコン側の実装条件	13
8	現在の maps / coefficients 研究候補例	14
9	実際の使い方	15
9.1	template package を育てる流れ	15
9.2	典型コマンド	15
9.3	Slurm での多忠実度探索と独立受入	15
9.4	ソーラーカー専用クリーン copy の作成	16
9.5	確認問題 3	17
10	まとめ	17

1 まず最初に全体像

1.1 一番上の流れ



1.2 PowerShell で叩くもの

action	mode	実体
up	sim / measure / live / live_wifi	build → stop → ros2 launch
build	任意	colcon build
status	任意	実行中プロセス確認
graph	任意	scripts/export_rqt_graph.py
simulate	sim 想定	scripts/solar_sim.py
forecast	live 前準備	scripts/fetch_weather_forecast.py
identify	identify	scripts/run_identification_pipeline.py
fit	full MLE	scripts/run_vehicle_identification.py
learn	planner 学習	scripts/tune_upper_planner_weights.py
audit	品質確認	inventory、static audit、pytest
package/blank	配布物	fitted 版 / 空テンプレート版を再生成

1.3 現在の source 側で可視になっている入口

今回の修正で、以前 source 側から見えなかった主要入口は、次の対応で source 可視化した。

source path	役割
launch/solar_measurement.launch.py	実測収集モードの ROS2 入口
launch/solar_race_live.launch.py	実運用 live モード入口
launch/solar_race_live_wifi.launch.py	WiFi 双方向運用入口
mpc_solarcar/solar_state_node.py	sim 中の車両状態 publisher
mpc_solarcar/speed_command_bridge_node.py	planner 指令の実機側 topic / UDP 橋渡し
mpc_solarcar/telemetry_text_bridge_node.py	ソーラーカー / 伴走車からの文字列テレメトリ受信
mpc_solarcar/wind_correction_node.py	風観測を用いた live forecast 補正
mpc_solarcar/weather_fetch_node.py	Open-Meteo 取得
mpc_solarcar/solar_autocal_node.py	solar / drive / aux の live 自動補正
mpc_solarcar/solar_profile.py	profile 読み込みと path 解決
mpc_solarcar/weather_utils.py	weather API と CSV 生成
scripts/solar_control.sh	PowerShell から呼ばれる WSL ルータ
scripts/export_rqt_graph.py	mode 別 graph export
scripts/fetch_weather_forecast.py	profile ベース forecast 作成
scripts/run_identification_pipeline.py	template / raw CSV ベースの識別入口

1.4 確認問題 1

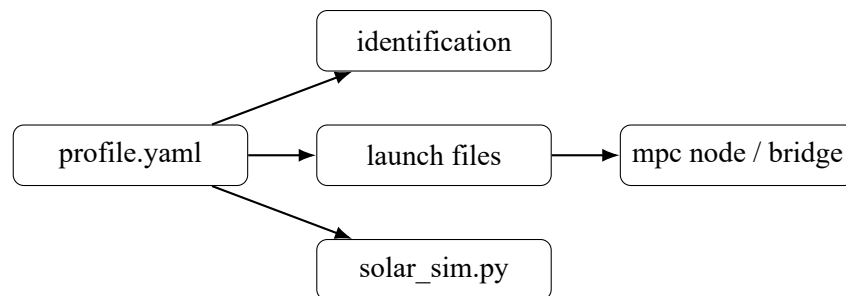
1. Windows 入口は **SolarSim.ps1** である。
2. WSL 側の振り分け本体は **scripts/solar_control.sh** である。
3. live_wifi で inbound / outbound 文字列通信を担う node は **telemetry_text_bridge_node** である。

2 profile.yaml の読み方

2.1 top-level 構造

block	意味
meta	package 名、目的、注記
paths	route, weather, map, schedule の実ファイル
runtime	dashboard host/port, forecast の時刻解釈
logging	共通 log 設定
simulation	offline sim の初期条件と出力先
measurement	distance / grade / logger の実測収集設定
identification	raw 入力と識別出力先
live	weather, autocal, command bridge, WiFi, wind model
model	質量, CdA, Crr, battery, PV, drive maps の係数
mpc	上位 / 下位 MPC の horizon, weight, constraint

2.2 この profile がどこに効くか



2.3 既定 profile

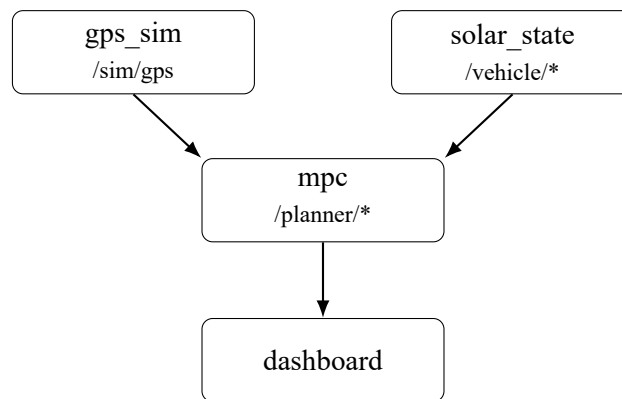
PowerShell の既定 path は `config/solar/bwsc_2027_demo.yaml` である。これは `project_packages/bwsc2027_template/` を参照する wrapper profile であり、空テンプレートを source 側から正しく叩けるようにするためのものである。したがって、

- build / forecast / identify の入口確認
- profile 構造確認

には使えるが、実走や full sim には、実データを入れた package profile を指定する必要がある。

3 mode 別に何が起きるか

3.1 sim



planner speed command は次周期の gps/state 伝播へ戻る。図では線の交差を避けるため feedback 線を省略した。

offline detail CSV では、`step\_dt\_sec` が下位指令周期で通常 1 秒である。`outer\_step\_requested\_dt\_sec=600` に対して `outer\_step\_actual\_dt\_sec` が 11.747 秒などになるのは、次の制御停止、走行時間窓、天候格子、上位速度区間の境界で外側ステップを正確に分割するためである。境界直前の `step\_dt\_sec` だけは 1 秒未満になり得るが、1 Hz 指令の欠落ではない。

- `gps_sim_node`: route waypoint 上を planner 指令速度で進め、`/sim/gps` を出す。
- `solar_state_node`: 同じ planner 指令速度から、`/vehicle/speed_kmh`, `/vehicle/s_km`, `/vehicle/batt_*`, `/vehicle/solar_power_w` を出す。
- `mpc_node`: profile の forecast / route / maps / model / mpc を読み、上位再計画と下位指令を出す。
- `dashboard_node`: 現在値と plan を 1 画面 API 化する。

3.2 measure

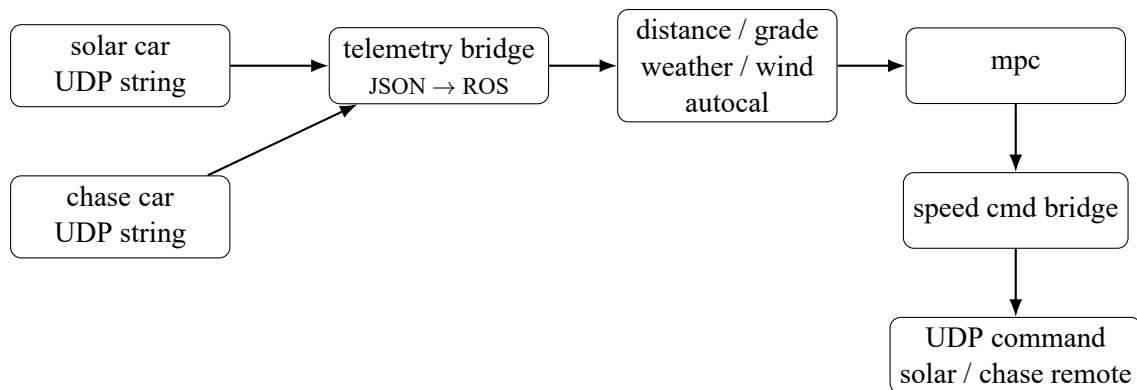
- `distance_node`: `/vehicle/speed_kmh` を積分して `/vehicle/s_km` を出す。
- `grade_node`: GPS / altitude / distance から勾配を出す。
- `solar_logger_node`: 電池、PV、風、planner、校正值、WiFi 生文字列、system 状態を同一時刻行の CSV にする。
- `dashboard_node`: 計測中の値を可視化する。

3.3 live

- `weather_fetch_node`: 現在の GPS または fallback 座標で Open-Meteo forecast を取得し、planner の読む CSV へ保存する。
- `mpc_node`: 1 時間再計画や 1 Hz lower command を行う本体。
- `solar_autocal_node`: 観測 solar / pack power と planner 予測の差から、`/calib/*` を publish する。
- `speed_command_bridge_node`: `/planner/speed_cmd` を安全な速度勾配に整形し、実車側の topic / UDP に渡す。

Open-Meteo archive は経路上の日射計による独立観測ではなく、model/reanalysis である。従って予報・欠測補間には使えるが、PV map の高精度認証を単独では行わない。no-trouble 方策探索は、地点・時刻ごとの archive 瞬時 GHI/DNI/DHI を保持する `bwsc2025_nominal_fullcourse_weather_grid.csv` だけを使う。`bwsc2025_historical_pv_conditioned_weather_grid.csv` は実車 PV から逆補正した循環的な診断専用品であり、方策順位付け、受入、live 入力には使わない。同定用には BOM Himawari hourly exposure を `scripts/import_bom_satellite_solar.py` で経路・時刻へ対応付ける。BOM または経路上 POA、同期 MPPT 状態が無い場合は `high_precision_gate_pass=false` のままとする。詳細監査は `outputs/reports/weather_power_timestep_audit/weather_power_timestep_audit.pdf` に保存する。

3.4 live_wifi



- `telemetry_text_bridge_node`: vehicle / chase からの JSON または key=value 文字列を受信し、ROS topic に展開する。
- `wind_correction_node`: 観測された向かい風から forecast CSV の `headwind_ms` を live 補正する。
- planner 側は修正済み forecast を見ながら upper/lower plan を更新する。

3.5 launch file ごとの実起動 node

以前は install 側にしか見えない source gap があったが、現在は launch / bridge / weather / profile utility を source 側へ戻してある。したがって、チームは次の表だけで mode ごとの立ち上がりを追える。

launch file	立ち上がる node	用途
<code>launch/solarcar_sim.launch.py</code>	<code>gps_sim_node</code> , <code>solar_state_node</code> , <code>mpc_node</code> , <code>dashboard_node</code>	offline sim と上位/下位 plan の確認
<code>launch/solar_measurement.launch.py</code>	<code>solar_preflight_node</code> , <code>dashboard_node</code> , <code>solar_logger_node</code> , optional <code>distance_node</code> , optional <code>grade_node</code>	実測収集と CSV 化
<code>launch/solar_race_live.launch.py</code>	<code>solar_preflight_node</code> , <code>mpc_node</code> , <code>dashboard_node</code> , <code>solar_logger_node</code> , <code>speed_command_bridge_node</code> , optional <code>distance_node</code> , <code>grade_node</code> , <code>weather_fetch_node</code> , <code>solar_autocal_node</code>	通常 live 運用
<code>launch/solar_race_live_wifi.launch.py</code>	live node 一式 + <code>telemetry_text_bridge_node</code> + optional <code>wind_correction_node</code>	WiFi 双方向運用

4 物理モデル

4.1 使っている基本式

車速 v , 向かい風成分 w , 勾配角 θ に対する機械出力の近似は、

$$P_{\text{mech}} = \left(\frac{1}{2} \rho C_d A (v + w)^2 + mg C_{rr} \cos \theta + mg \sin \theta \right) v$$

である。補機は駆動効率 `map` を通さず、電気側で直接加える。走行中と日中停車は $P_{\text{aux}} = P_{\text{aux,stopped}} = 21.021 \text{ W}$ 、日射が `YAML` の 20 W/m^2 閾値以下となる夜間だけ $P_{\text{aux,night}} = 0 \text{ W}$ である。

なお、運用シミュレーション初期値 `simulation.soc0=0.98` と、履歴ログ再生だけに用いる潜在推定値 `identification.fitted_replay_soc0=0.966219` は別の状態である。後者を前者へ書き戻してはならない。同定 `writer` は既定で両者を分離し、回帰テストでもこの契約を固定している。

PV は、POA 日射 G_{poa} 、セル温度 T_c 、パネル効率 `map`、MPPT 効率 `map` を用いて

$$P_{\text{pv}} = \eta_{\text{panel}}(G_{\text{poa}}, T_c) \eta_{\text{mppt}}(G_{\text{poa}}, T_c) A_{\text{pv}} G_{\text{poa}}$$

とする。

実車が WiFi で送る `solar_power_w` は、補正前の DC bus 実測値 $P_{\text{solar,raw}}$ とする。日中停車かつ駆動電力がほぼゼロの標本だけを使い、実測バッテリー電力との関係

$$P_{\text{batt},i} = P_{\text{aux}} - g_{\text{solar}} P_{\text{solar,raw},i} + \varepsilon_i$$

から、外れ値に強い Huber M 推定

$$(\hat{g}_{\text{solar}}, \hat{b}) = \arg \min_{g,b} \sum_i \rho_\delta(P_{\text{batt},i} - P_{\text{aux}} + g P_{\text{solar,raw},i} - b)$$

を解く。 g は物理範囲内へ拘束し、切片 b 、日別推定値の標準偏差、採用標本数も同定 `gate` へ入れる。送信側は `raw` 値を送り、受信側が $P_{\text{solar}} = \hat{g}_{\text{solar}} P_{\text{solar,raw}}$ を一度だけ適用する。送信側でも補正すると二重補正になるため禁止する。

パック出力は

$$P_{\text{pack}} = P_{\text{drive,dc}} - P_{\text{regen,dc}} + P_{\text{aux}} - P_{\text{pv}}$$

であり、SoC 更新は

$$z_{k+1} = z_k - \eta_{\text{chg}}(I_k) \frac{I_k \Delta t_k}{3600 Q_{\text{nom}}}$$

$$V_{p,k+1} = e^{-\Delta t / \tau_p} V_{p,k} + (1 - e^{-\Delta t / \tau_p}) R_p I_k, \quad V_k = \text{OCV}(z_k) - I_k R_0 - V_{p,k}$$

である。

YATA の電池は 25 直列で、`race-use` 上限を 4.35 V/cell とするため、

$$V_{\text{max}} = 25 \times 4.35 = 108.75 \text{ V}$$

である。OCV `map` の最大値はこの上限以下でなければならない、`profile` の `model.V\max` は OCV `map` 最大値以上かつ 108.75 V に一致しなければならない。OCV `map` がこの上限を超えた `fit` も不採用とする。この契約は静的監査と回帰テストの双方で検査する。

現行 MLE35 候補の `battery-conditioned replay` は、clean 電力 RMSE 201.344 W 、clean 電圧 RMSE 0.968 V である。天候・PV 予測も含む `end-to-end replay` は、それぞれ 246.088 W 、 2.113 V 、 25 km energy RMSE は 102.944 Wh である。`battery-conditioned` 終端 SoC 誤差は 0.006821 まで縮小したが、 2831 km 終端 SoC の独立証拠は 0.332236 から 0.528667 まで $19.643 \text{ percentage points}$ 食い違う。従って同期 BMS coulomb count、rested OCV/温度、MPPT 状態、POA 日射が揃うまで `high_precision_gate_pass=false` とする。終端一点の一致だけをモデル高精度の証明にしてはならない。

旧 4 変数方策 $[70, 70, 70, 67] \text{ km/h}$ は `legacy` 比較に限る。現行探索は全 3026.9 km に対し、積分格子を 5 km 、 1 km 、 0.1 km へ細分し、速度制御間隔を 25 km 、 5 km 、 2 km 、 1 km へ細分する。制御次元は `finish`

点を含めて 123、607、1515、3028 である。4 独立 seed、8400 世代、32,993,280 候補を CUDA CEM で提案し、各最終方策を別の 1 Hz CPU simulator で再計算する。その後、固定方策の 1/0.5/0.2/0.1 km 積分格子と、独立最適化した 5/2/1 km 制御方策の収束を判定する。CEM の最良値は連続領域全体の数学的な大域最適性証明ではなく、exact replay、格子収束、車両 model 精度は別 gate である。

向かい風成分は風速 u 、風向 ϕ 、進行方位 ψ から

$$w_{\text{head}} = u \cos(\phi - \psi)$$

で作る。

4.2 実装でこれを使っている場所

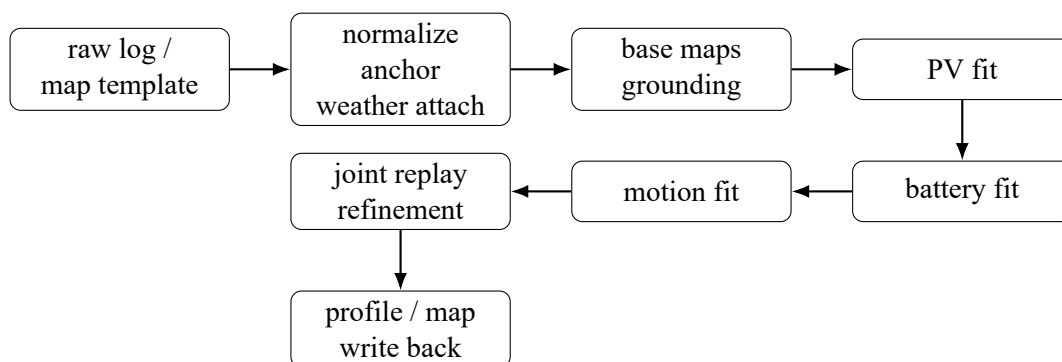
- `mpc_solarcar/model.py`: 抵抗・PV・battery IV・SoC 更新
- `mpc_solarcar/mpc_node.py`: 上位 / 下位 MPC 評価
- `mpc_solarcar/upper_policy.py`: 受入済み速度方策の検証・補間と上位 MPC 初期解生成
- `scripts/solar_sim.py`: offline 全体 simulation
- `mpc_solarcar/solar_state_node.py`: sim 中の realtime 状態再生

4.3 確認問題 2

1. SoC 更新の分母に入る nominal 容量は E_{nom} である。
2. 向かい風成分は $u \cos(\phi - \psi)$ で与える。

5 実測ログからの MLE / fitting

5.1 pipeline



5.2 軽量 pipeline

`scripts/run_identification_pipeline.py` は、

- `gps_track.csv`
- `battery_rest.csv`
- `battery_pulse.csv`
- `panel_sweep.csv`
- `drive_timeseries.csv`

から route / OCV / Rint / PV maps / vehicle scalar を構築する軽量 pipeline である。

5.3 本 package の本格 replay fit

`scripts/run_vehicle_identification.py` は、正規化済み replay log と既存 maps を使って、

$$\begin{aligned}\mathcal{L}_{PV}(\theta_{PV}) &= \sum_k \rho \left(P_{pv,k}^{obs} - P_{pv,k}^{pred}(\theta_{PV}) \right), \\ \mathcal{J}_B(\theta_B) &= \mathcal{L}_B(\theta_B) + \mathcal{R}_B(\theta_B), \\ \mathcal{J}_M(\theta_M) &= \mathcal{L}_M(\theta_M) + \mathcal{R}_M(\theta_M),\end{aligned}$$

を順に最小化して PV, battery, motion を詰める。

ルート勾配は車両係数へ吸収せず、観測モデルとして別に検証する。DEM 高度 $h(s)$ に Savitzky–Golay 平滑化を施した $\tilde{h}_L(s)$ から

$$q_{L,\delta}(s) = 100 \frac{d\tilde{h}_L(s + \delta)}{d(1000s)}$$

を作り、平滑化長 L 、距離オフセット δ 、仮の勾配倍率を Day1–Day5 の電力 RMSE で選ぶ。Day6 を選択に使わず、同日の RMSE を悪化させない候補だけを採択する。出力する `adopted\_route\_profile.csv` は未スケールの $q_{L,\delta}$ を保持し、 γ_{grade} はその後の motion fit で別に再同定する。summary には候補数、選択した L, δ 、train/holdout RMSE をすべて保存する。

ここで、

- θ_{PV} : panel gain, cell temperature gain
- θ_B : $z_0, E_{\text{nom}}, k_R, R_{\text{line}}, \eta_{\text{charge}}$
- θ_M : $C_d A, C_{rr}, P_{\text{aux}}, \gamma_{\text{grade}}, k_\eta, k_w$

である。

5.4 fit した値はどこへ入るか

- map そのもの: `profile.paths.*` が指す CSV
- scalar coefficient: `profile.model.*`
- planner weight / horizon: `profile.mpc.*`
- offline sim: `scripts/solar_sim.py` が `profile` から直読
- live: launch file が `profile` を展開し、`mpc_node` / `bridge` / `weather node` に渡す

5.5 どのファイルを入れれば generic fitting が回るか

template file	役割
templates/identification/ observed_replay_log_ template.csv	正規化 replay log の雛形
templates/identification/ gps_track_template.csv	route reconstruction の雛形
templates/identification/ panel_sweep_template.csv	panel / MPPT map
templates/identification/ battery_rest_template.csv	OCV
templates/identification/ battery_pulse_template.csv	Rint
templates/identification/ drive_timeseries_template. csv	vehicle scalar fit
templates/identification/ source_map_catalog_template. csv	使用した元資料の整理

6 一画面 dashboard の読み方

図1は、実際の dashboard/ を教育用固定テレメトリで 1920×1080 描画したものである。全カードと system status がスクロールなしで収まることを目視確認した。固定値は scripts/dashboard_demo_server.py が返すため、実車値ではない。

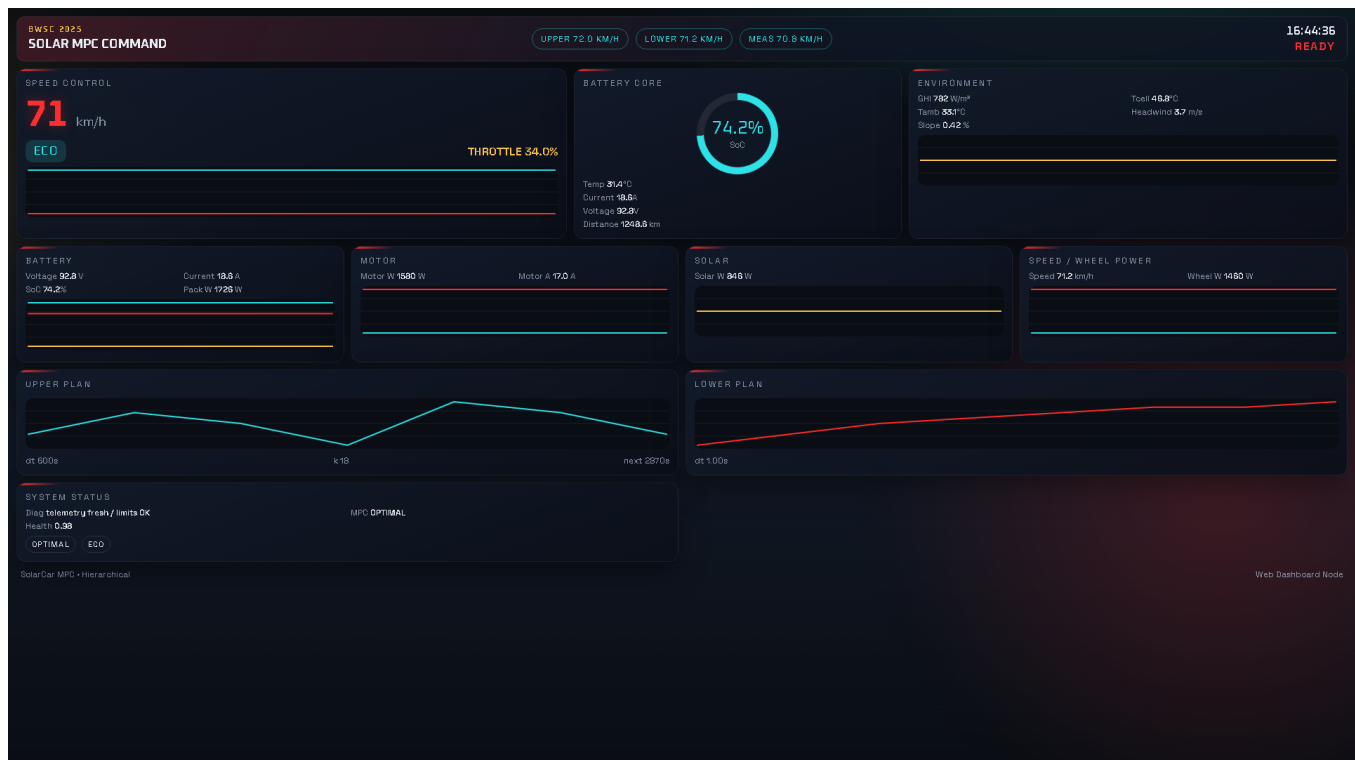


Figure 1: 実 dashboard の一画面デモ。上から現在指令、車両・環境、電力、上位・下位 plan、健全性を並べる。

- 上端の UPPER / LOWER / MEAS は、上位計画、1 Hz 下位指令、実測速度である。差が継続すれば追従異常を疑う。
- SPEED CONTROL は最終速度指令、drive mode、throttle 相当量を示す。実機へ送る値は rate limit 後の値である。
- BATTERY CORE と BATTERY は SoC、温度、電圧、電流、pack 電力、距離を示す。SoC だけでなく電圧・電流制約を同時に見る。
- ENVIRONMENT は POA 日射、セル・外気温、補正向かい風、勾配であり、上位予測の主要外乱である。
- MOTOR / SOLAR / SPEED-WHEEL POWER は、駆動、発電、車輪機械出力を分け、符号・効率 map の異常を切り分ける。
- UPPER PLAN は残距離の速度計画、LOWER PLAN は直近数秒の追従計画である。両者の時間尺度を混同しない。
- SYSTEM STATUS は telemetry freshness、limit、MPC solve 状態、health を示す。READY でも diag を必ず読む。

教育用画面は repository root で

```
python scripts/dashboard_demo_server.py --port 18080
```

を実行し、<http://127.0.0.1:18080> を開く。実運用ではこのデモを起動せず、dashboard_node.py の /api/state を使用する。

7 live_wifi の送受信仕様

7.1 推奨方式

推奨は UTF-8 の UDP JSON である。newline は不要、1 パケット 1 JSON を基本とする。全 Raspberry Pi は起動前に NTP / chrony で UTC 同期する。既定では timestamp_utc または ts_unix が必須であり、5 秒より古い packet、2 秒より未来の packet、重複・時刻逆転 packet は filter や ROS topic 更新の前に破棄される。

7.2 ソーラーカー側から送る推奨 JSON

```
{
  "type": "vehicle_state",
  "ts_unix": 1762500000.0,
  "speed_kmh": 72.3,
  "soc": 0.83,
  "batt_temp_c": 34.2,
  "batt_current_a": 8.5,
  "batt_voltage_v": 97.6,
  "solar_power_w": 410.0,
  "lat": -19.13542,
  "lon": 146.81235,
  "alt_m": 18.4,
  "wind_speed_ms": 6.8,
  "wind_dir_deg": 118.0,
  "course_deg": 176.5
}
```

7.3 伴走車側から送る推奨 JSON

```
{
  "type": "chase_state",
  "ts_unix": 1762500001.0,
  "speed_kmh": 74.0,
  "lat": -19.13580,
  "lon": 146.81210,
  "alt_m": 18.1,
  "wind_speed_ms": 6.5,
  "wind_dir_deg": 121.0,
  "course_deg": 176.0
}
```

7.4 planner 側から返す JSON

```
{
  "type": "planner_command",
  "ts_unix": 1762500002.0,
  "timestamp_utc": "2025-11-07T10:00:02Z",
  "planner": {
    "speed_cmd_kmh": 68.0,
    "upper_speed_cmd_kmh": 69.0,
    "drive_mode": "eco"
  },
  "vehicle": {
    "speed_kmh": 67.4,
    "soc": 0.81,
    "s_km": 412.3
  }
}
```

7.5 key=value fallback

telemetry_text_bridge_node.py は JSON だけでなく

type=vehicle_state,speed_kmh=71.8,soc=0.82,batt_voltage_v=97.4

のような key=value も構文として扱えるが、通常運用では同じ列に timestamp_utc=...Z または ts_unix=... を必ず付ける。

7.6 マイコン側の実装条件

- WiFi: 同一 LAN, UDP
- 文字コード: UTF-8
- 推奨周期: 1 Hz
- 起動前に NTP / chrony 同期を確認し、送信 packet に毎回 UTC 時刻を入れる
- パケット喪失を考え、planner command は 3 秒 timeout で無効化
- 受信未更新時はマイコン側でも fail-safe を持つ
- 参考: scripts/wifi_vehicle_sender_example.py

- 参考: scripts/wifi_chase_sender_example.py
- 参考: templates/network/esp32_planner_receiver_example.ino

8 現在の maps / coefficients 研究候補例

現時点で追跡しやすい fitted 研究候補は次の package である。

project_packages/bwsc2025_fitted_mle19_energywindow_inertia/

ここで示す MLE35 は immutable な未昇格研究候補であり、canonical live profile ではない。本番用 profile_operational_fine.yaml は $C_d A = 0.109802$ 、 $C_{rr} = 0.006560$ 、 $P_{aux} = P_{aux,stopped} = 20.891\text{ W}$ 、 $P_{aux,night} = 0\text{ W}$ 、 $E_{nom} = 2899.987\text{ Wh}$ 、 $Q_{nom} = 31.783316\text{ Ah}$ を保持している。旧値の方が正確という意味ではなく、独立モデル検証ゲートを通らない研究候補を本番へ混入させないための分離である。

この MLE35 候補の current_maps_and_coefficients.md 要約を参照すると、候補内で評価する asset は次である。

asset	役割
outputs/identification/runs/mle35_expanded_grade_single_source_ultra_v1/adopted_maps/drive_eff_map.csv と eco/power 派生	駆動効率 map
outputs/identification/runs/mle35_expanded_grade_single_source_ultra_v1/adopted_maps/regen_eff_map.csv と eco/power 派生	回生効率 map
outputs/identification/runs/mle35_expanded_grade_single_source_ultra_v1/adopted_maps/Rint_T_by_soc_fitted_grounded.csv	温度・SoC 依存内部抵抗 map
outputs/identification/runs/mle35_expanded_grade_single_source_ultra_v1/adopted_maps/panel_eff_map.csv	パネル効率 map
outputs/identification/runs/mle35_expanded_grade_single_source_ultra_v1/adopted_maps/mppt_eff_map.csv	MPPT 効率 map
outputs/identification/runs/mle35_expanded_grade_single_source_ultra_v1/adopted_maps/ocv_soc_curve.csv	OCV-SoC 曲線
data/weather/bwsc2025_nominal_fullcourse_weather_grid.csv	大会全体 nominal weather grid
data/observed/bwsc2025_observed_log_5s.csv	progress reference / replay log

同要約に出ている代表 scalar は、

$$m = 235.0, \quad C_d A = 0.111167, \quad C_{rr} = 0.006262, \quad P_{\text{aux}} = 21.021 \text{ W},$$

$$\begin{aligned} P_{\text{aux,stopped}} &= 21.021 \text{ W}, & P_{\text{aux,night}} &= 0 \text{ W}, \\ E_{\text{nom}} &= 3011.000 \text{ Wh}, & Q_{\text{nom}} &= 33.000 \text{ Ah}, \\ R_p &= 0.084252 \Omega, & \tau_p &= 73.595 \text{ s}, \\ \text{panel_gain} &= 0.712687, & \text{drive_eff_scale} &= 1.034296, \\ \text{solar_measurement_gain} &= 0.922613, & \text{rint_scale} &= 0.900000. \end{aligned}$$

である。

no-trouble 方策評価はトラブル停車を除くが、公式 control stop を除かない。data/race/bwsc2025_official_control_stops.yaml に 9 地点、各 1800 秒、公式開閉時刻、Adelaide 締切 2025-08-29T08:00:00Z を保存する。早着時は開場まで待機し、control stop 閉鎖後または finish 締切後の到着は、CUDA surrogate と厳密 1 Hz replay の両方で不成立とする。時刻根拠は <https://assets.worldsolarchallenge.org/app/uploads/2025/08/20102713/BWSC-2025-Official-Program-v4.pdf> の control-stop 表である。17:30 には Cruiser Class 最終到着との脚注があるため、報告書ではこの注記も保持する。

9 実際の使い方

9.1 template package を育てる流れ

1. project_packages/bwsc2027_template/ に route, weather, map, schedule, identification raw を入れる。
2. python scripts/run_identification_pipeline.py --profile_yaml <profile> で軽量 map 作成を行う。
3. replay log があるなら python scripts/run_vehicle_identification.py --profile <profile> で本格 fit を行う。
4. powershell -ExecutionPolicy Bypass -File .\SolarSim.ps1 で full sim を確認する。
-Action simulate -Profile <profile>
5. powershell -ExecutionPolicy Bypass -File .\SolarSim.ps1 で実運用へ入る。
-Mode live_wifi -Action up -Profile <profile>

9.2 典型コマンド

```
powershell -ExecutionPolicy Bypass -File .\SolarSim.ps1 -Action build
powershell -ExecutionPolicy Bypass -File .\SolarSim.ps1 -Action forecast
powershell -ExecutionPolicy Bypass -File .\SolarSim.ps1 -Action simulate `
-Profile <profile>
powershell -ExecutionPolicy Bypass -File .\SolarSim.ps1 -Mode live_wifi -Action up `
-Profile <profile>
python scripts/run_vehicle_identification.py --profile <profile>
```

9.3 Slurm での多忠実度探索と独立受入

GPU CEM は多数方策を提案し、CPU 側は固定方策 1 Hz 再計算と格子収束だけを受入根拠にする。GPU 探索は 1 割当内で 4 seed を直列化する。環境変数 REPLICATE_INDEX=0..3 を別 job へ与えた場合だけ、scheduler が GPU quota の範囲で並列化する。1 GPU 内へ 4 CUDA process を競合配置せず、受入側の CPU 処理には後述の最大 12 並列を用いる。

```

sbatch --export=ALL,SOLAR_GPU_ROOT=$PWD,\
PYTHON_BIN=$HOME/.venvs/mpc_gpu/bin/python \
scripts/run_solar_gpu_concurrent_campaign.sbatch <profile> <campaign_dir>

```

```

sbatch --dependency=afterok:<job_id> --export=ALL,SOLAR_GPU_ROOT=$PWD,\
PYTHON_BIN=$HOME/.venvs/mpc_gpu/bin/python \
scripts/run_solar_gpu_acceptance_pipeline.sbatch \
<profile> <campaign_dir> <acceptance_dir>

```

独立ジョブ方式ではcampaign_submission.yaml の finalize_job_id を次の依存指定に使う。

```

sbatch --dependency=afterok:<finalize_job_id> ...

```

finalizer は 4 本すべての control_1km/latest_policy.csv を確認してから CAMPAIGN_COMPLETE を書く。また、投入時に profile の SHA-256 を campaign_submission.yaml へ保存し、coarse、fine、ultra、control-refine、finalize の全 stage が開始前に同じ hash を再計算する。探索中に YAML が変更された場合は、その stage を失敗させる。これにより異なる車両モデルの評価値を同じ収束履歴へ混在させない。

受入ジョブは 5、2、1 km の 3 格子を並列実行し、各格子内の 4 seed も並列実行する。従って要求した 16 CPU のうち最大 12 CPU を用いて、固定 policy の 100 m 予測と 1 Hz 全行程再生を行う。各 seed は固有の profile、manifest、detail CSV、判定表を持ち、全 worker 終了後にだけ最速の可行 seed を選択する。1 seed の未可行は候補棄却であり、出力欠損または全 seed 未可行の場合だけその stage を失敗とする。

単一 GPU 割当方式では 4 本の PIPELINE_COMPLETE、独立ジョブ方式では 4 本の最終 policy、さらに CAMPAIGN_COMPLETE、ACCEPTANCE_COMPLETE を確認する。exact 候補が無い、または格子収束が未達なら ACCEPTANCE_FAILED となり、live profile へ昇格してはならない。exact 1 Hz replay と mesh 収束だけが通った段階では POLICY_ACCEPTANCE_COMPLETE と profile_exact_selected_research.yaml だけを作る。独立車両 model gate も通った場合に限り ACCEPTANCE_COMPLETE、profile_exact_selected.yaml、profile_live_mpc_learned.yaml を作る。従って POLICY_ACCEPTANCE_COMPLETE と ACCEPTANCE_FAILED の併存は、方策計算は合格したが車両 model を実運用認定できないことを表す。制御格子は 5 km、2 km、1 km で独立に最適化した policy を使う。最細分の --policy は必ず 1 km へ割り当て、2 km policy を上書きしない。受入成功時は二つの profile を生成する。profile_exact_selected.yaml は方策を固定した 1 Hz 再現試験専用である。profile_live_mpc_learned.yaml は同じ CSV を paths.initial_upper_policy_csv へ設定するが固定しない。mpc_node.py は起動時にこの方策を現在の制御格子へ補間し、上位 MPC の初期解としてだけ使用する。その後は実測 SoC、現在距離、更新予報、制約を使って上位問題を再度解き、次回からは直前の MPC 解を移動後の格子へ補間する。従って GPU CEM は offline proposer、live 指令の最終決定権は receding-horizon MPC にある。全成果物の日本語 TeX/PDF は次で作る。

```

python scripts/generate_gpu_acceptance_report.py \
--profile <profile> --campaign-dir <campaign_dir> \
--acceptance-dir <acceptance_dir> --fit-summary <fit_summary> \
--promotion-gate <promotion_gate.json> --output-dir <report_dir>

```

この報告書は GPU 収束履歴、全 exact 順位、1 Hz fullsim、積分・制御格子収束、独立モデル gate を統合する。

9.4 ソーラーカー専用クリーン copy の作成

- python scripts/create_solarcar_only_package.py を実行すると、配布用の整理済み --output_dir exports/solarcar_only_package_20260712 copy を生成する。
- copy 側では passo_* launch、passo_run.sh、磁力関連 source / script、旧 bwsc2025_fitted_mle* package、build / install / log / outputs を含めない。

- さらに copy 内の `setup.py` から磁力 entry point を除去し、`mpc\_node.py` は solar 専用初期化に固定する。
- 配布相手には、この clean copy と `docs/solar_all_in_one_manual/solar_all_in_one_manual.pdf` をそのまま渡せばよい。

9.5 確認問題 3

1. 軽量 map 作成の入口は `scripts/run_identification_pipeline.py` である。
2. full-race offline sim の入口は `scripts/solar_sim.py` である。
3. live で Open-Meteo を取りに行く node は `weather_fetch_node` である。

10 まとめ

- 入口は `SolarSim.ps1` → `scripts/solar_control.sh` である。
- mode ごとの launch と node は source 側で追える状態へ戻した。
- profile は sim / live / identify の唯一の接着剤であり、maps と scalar の反映先でもある。
- MLE / replay fit は、PV, battery, motion, joint replay を順に詰め、その結果を profile と maps に書き戻す。
- live_wifi では、JSON 文字列テレメトリ → ROS topic → weather / wind / autocal → MPC → speed command bridge という一直線の流れで理解するのが最も見通しが良い。